

KernelBench v3: Rebuilding a GPU Kernel Benchmark from First Principles

Elliot Arledge

January 26, 2026

Abstract

Efficient GPU kernels are essential for modern ML systems, but kernel engineering remains expensive and specialized. KernelBench v3 is a benchmark for evaluating whether large language models can produce optimized CUDA kernels that are both correct and faster than established PyTorch baselines. We rebuild KernelBench with a smaller, higher quality task set focused on modern architectures, and we report results across 9 models, two GPUs, and 41 problems. Across 738 evaluations, 42.4% pass correctness checks and 19.8% beat the baseline. The most capable models can beat the baseline on many Level 1 and 2 tasks, but performance collapses on Level 4 problems that use novel architectures or already-optimized Triton baselines. These results highlight the gap between code generation and production-grade kernel engineering.

1 Introduction

Kernel engineering is a bottleneck for deploying efficient ML systems. While modern compilers and libraries provide optimized kernels, tailoring kernels to new architectures or fused operators often requires expert attention. Recent work introduced KernelBench to test whether LLMs can automate this process by producing faster CUDA kernels from PyTorch reference code [10]. The benchmark sparked interest, but subsequent analyses showed that benchmark design and evaluation details strongly influence results [4].

KernelBench v3 is a rebuild focused on modern inference kernels, stronger baselines, and transparent cost accounting. The benchmark reduces the problem set to 41 tasks that are representative of current workloads, prioritizes Hopper and Blackwell GPUs, and tracks per-run tokens and cost. The goal is not to maximize speedups with fragile tricks, but to measure whether models can produce kernels that compile, are numerically correct, and outperform real baselines.

This paper makes three contributions:

- A curated benchmark of 41 modern kernel tasks across four difficulty levels.
- An evaluation pipeline that reports compilation, correctness, and baseline-beating rates, with full per-run cost accounting.
- A results summary across 9 models on H100 and B200 GPUs, highlighting where current models still fail.

2 Background and Motivation

KernelBench v0.1 and the original KernelBench paper established a useful framing: models are given a PyTorch reference implementation and must produce a faster CUDA kernel [10, 11]. Subsequent investigations revealed several reward-hacking patterns, including calling high-level libraries instead of writing kernels, manipulating timing, or exploiting numerically trivial tasks [4]. These findings motivated a ground-up redesign rather than incremental fixes.

KernelBench v3 is designed with two principles. First, tasks should reflect modern kernel engineering needs, including fused matmul chains, attention blocks, and emerging linear-attention variants. Second, baselines

Table 1: KernelBench v3 levels and turn limits.

Level	Tasks	Turns	Description
L1	15	10	Simple ops: matmul, softmax, conv, norms
L2	15	12	Fused ops: matmul and activation chains
L3	3	15	Single blocks: attention and transformer block
L4	8	15	Novel layers: MLA, MoE, GQA, FP8, INT4, DeltaNet

should be meaningful: for some Level 4 tasks, the baseline is already an optimized Triton implementation, so beating the baseline requires genuinely competitive kernel engineering.

This redesign also emphasizes cost realism. Running large model sweeps across many problems and hardware targets can become prohibitively expensive. KernelBench v3 focuses on two widely deployed data center GPUs and a smaller, curated task set while preserving the evaluation challenge.

3 Benchmark Design

KernelBench v3 contains 41 tasks across four levels (Table 1). Each task provides a PyTorch reference model and an input generator. The model under evaluation must produce a CUDA implementation that matches the reference output and improves runtime. Level 1 contains basic kernels such as matmul, softmax, and normalization. Level 2 contains fused multi-op patterns such as matmul-plus-activation-plus-norm chains. Level 3 covers single attention blocks and a transformer block. Level 4 focuses on modern inference kernels, including DeepSeek-style MLA and MoE patterns [3], GQA [1], FP8 and INT4 matmuls [5, 14], and linear-attention variants such as Gated DeltaNet and Kimi Delta Attention [12, 15].

Some Level 4 baselines are already optimized Triton implementations [13]. For example, the Gated DeltaNet and Kimi Delta Attention tasks use optimized kernels rather than naive PyTorch baselines, which sets a high bar for LLM-generated code. As a result, Level 4 performance reflects whether models can compete with production-quality kernels rather than only outperform naive baselines.

KernelBench v3 evaluates on two target GPUs: H100 (Hopper) and B200 (Blackwell) [7, 9]. The benchmark runs in a Modal sandbox with CUDA 13.1 [6, 8], providing a consistent environment for compilation and benchmarking.

4 Evaluation Methodology

Each evaluation is a multi-turn agent loop. The model receives a PyTorch reference implementation, GPU target, and constraints, then iteratively proposes CUDA code. We cap the number of turns per problem by level (Table 1) to control cost.

Compilation. A submission is marked *compiled* if the generated code defines the required model class and executes without errors in the Modal sandbox [6].

Correctness. A compiled kernel is considered correct if its output matches the reference model within a tolerance. The evaluation script computes the maximum absolute error and compares it to `atol + rtol * max(|ref|)` with `atol=0.05` and `rtol=0.02`.

Performance. We time the reference model and the generated kernel over 10 iterations with CUDA synchronization, after a short warmup. Speedup is defined as the reference runtime divided by the candidate runtime. A kernel *beats the baseline* when its speedup exceeds 1.0.

Metrics. We report compilation rate, correctness rate, and baseline-beating rate. For each run, we also log tokens and estimated API cost, enabling cost-adjusted analysis.

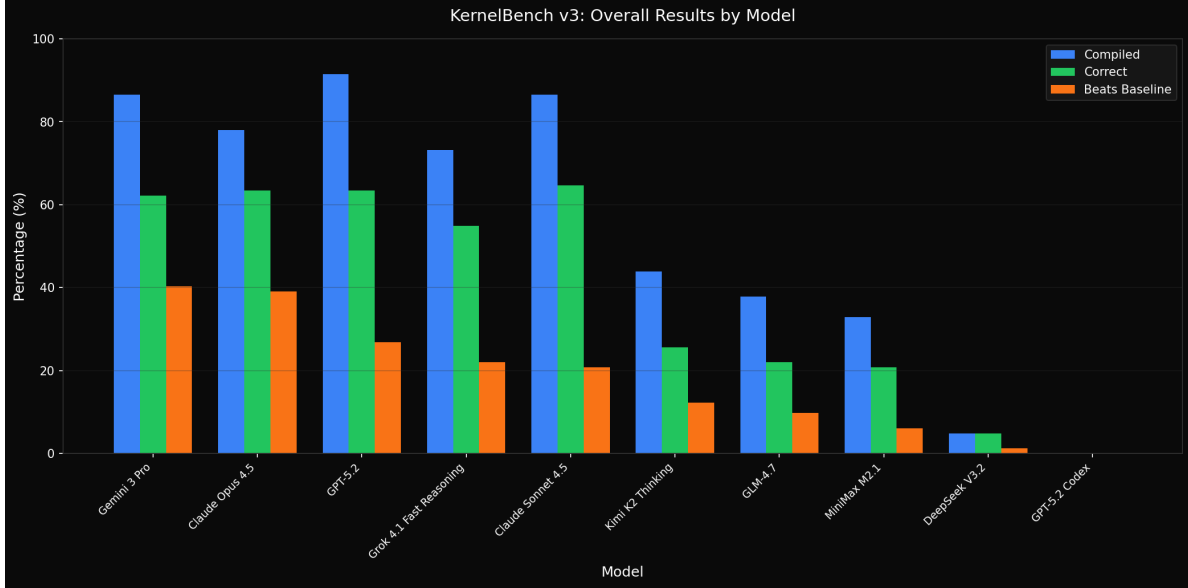


Figure 1: Overall results by model. Bars show the fraction of runs that compiled, were correct, and beat the baseline.

Table 2: Model-level results across all problems and GPUs.

Model	Correct (%)	Beat baseline (%)	Beat baseline (n)	Total
Gemini 3 Pro	62.2	40.2	33	82
Claude Opus 4.5	63.4	39.0	32	82
GPT-5.2	63.4	26.8	22	82
Grok 4.1 Fast Reasoning	54.9	22.0	18	82
Claude Sonnet 4.5	64.6	20.7	17	82
Kimi K2 Thinking	25.6	12.2	10	82
GLM-4.7	22.0	9.8	8	82
MiniMax M2.1	20.7	6.1	5	82
DeepSeek V3.2	4.9	1.2	1	82

5 Results

We report results over 738 evaluations covering 9 models, two GPUs, and 41 problems. Overall, 42.4% of runs pass correctness and 19.8% beat the baseline. The average speedup among correct kernels is 1.522x over 313 successful runs.

Figure 1 is sourced from the public dashboard export. GPT-5.2 Codex appears at 0% there because the evaluation failed due to an API format mismatch in the OpenAI Codex endpoint, not model performance. We exclude GPT-5.2 Codex from the tables and aggregate statistics.

Table 2 summarizes performance by model. Gemini 3 Pro and Claude Opus 4.5 produce the most baseline-beating kernels. GPT-5.2 has strong correctness but fewer baseline wins, indicating that correctness alone does not guarantee competitive performance.

The figures are sourced from the public dashboard export. GPT-5.2 Codex appears at 0% there because the evaluation failed due to an API format mismatch in the OpenAI Codex endpoint, not model performance. We exclude GPT-5.2 Codex from the tables and aggregate statistics.

Table 3 breaks down performance by difficulty level. Pass rates decline sharply at Level 4: only 11.8% of runs pass correctness and 4.2% beat the baseline. These tasks include modern attention variants and quantized matmuls, and some use already-optimized Triton baselines.

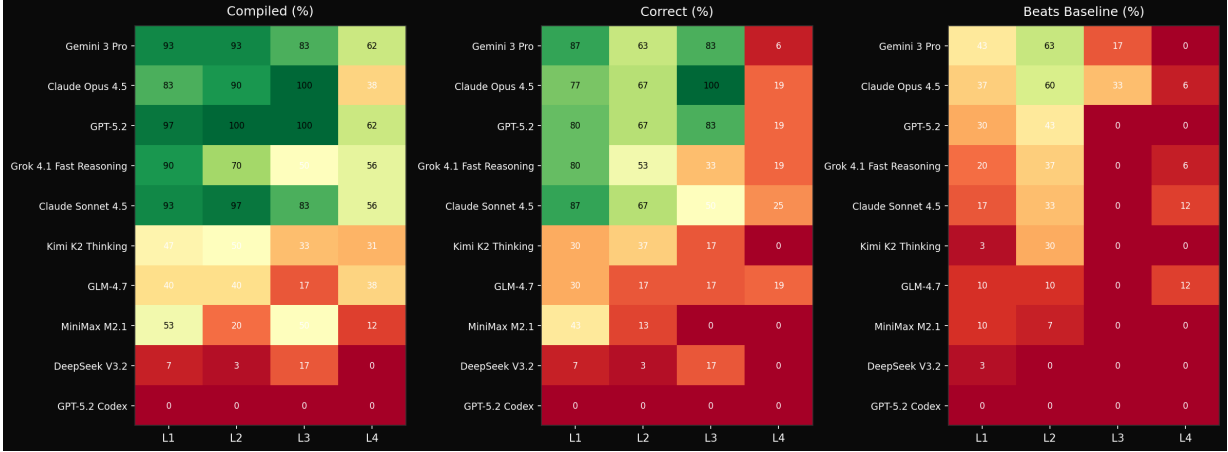


Figure 2: Heatmap of correctness and baseline-beating rates by level.

Table 3: Results by difficulty level.

Level	Pass rate (%)	Beat baseline (%)	Beat baseline (n)	Total
L1	57.8	19.3	52	270
L2	43.0	31.5	85	270
L3	44.4	5.6	3	54
L4	11.8	4.2	6	144

6 Discussion

KernelBench v3 results reinforce two observations. First, baseline-beating kernels are rare even for top models. This aligns with METR’s finding that kernel benchmarks can overstate capability when task design is weak [4]. Second, performance collapses on Level 4 tasks, which emphasize modern attention and quantization patterns [5, 14]. These tasks are not just harder because of novelty but also because some baselines are already optimized Triton implementations [13], shrinking the remaining optimization headroom.

We also observe that correctness is not the limiting factor for all models. Several models achieve respectable correctness rates but lower baseline-beating rates, suggesting that the remaining gap is performance engineering rather than syntax or functional accuracy. For practical kernel engineering, models must reason about memory layout, kernel fusion, and hardware-specific bottlenecks.

Finally, cost remains a key practical constraint. The benchmark logs per-run token counts and estimated costs, enabling analysis of accuracy and speedup as a function of spend. This makes KernelBench v3 useful for evaluating cost-performance tradeoffs, not just raw capability.

7 Limitations and Future Work

KernelBench v3 is limited to single-GPU kernels and does not yet cover multi-GPU collective patterns. Correctness is evaluated on a single input per run with a fixed tolerance, which may miss some numerical edge cases. The benchmark also uses a fixed number of turns per task; some models might achieve better performance with longer optimization loops.

Future work includes adding multi-GPU tasks, expanding the Level 4 suite as new architectures emerge, and strengthening correctness checks with multi-seed or distributional testing. An open question is how to design evaluation protocols that reward robust kernel engineering without encouraging benchmark-specific shortcuts.

8 Conclusion

KernelBench v3 offers a focused, modern benchmark for evaluating whether LLMs can produce optimized GPU kernels. Across 738 runs, only 19.8% of evaluations beat the baseline, and Level 4 performance remains low. These results suggest that current models are helpful assistants for kernel development but are not yet autonomous kernel engineers. The benchmark and results are released to support ongoing research into reliable, performant code generation [2].

Acknowledgements

Thanks to the open-source kernel and inference community for their feedback and tooling. Special thanks to the KernelBench and METR teams for publicly documenting pitfalls and evaluation methodology.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL <https://arxiv.org/abs/2305.13245>.
- [2] Elliot Arledge. Kernelbench v3 repository. <https://github.com/Infatoshi/KernelBench-v3>, 2026.
- [3] DeepSeek-AI et al. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- [4] METR. Measuring automated kernel engineering. <https://metr.org/blog/2025-02-14-measuring-automated-kernel-engineering/>, 2025.
- [5] Paulius Micikevicius et al. Fp8 formats for deep learning, 2022. URL <https://arxiv.org/abs/2209.05433>.
- [6] Modal Labs. Modal documentation. <https://modal.com/docs>, 2024.
- [7] NVIDIA. Nvidia hopper architecture whitepaper. <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>, 2022.
- [8] NVIDIA. Cuda toolkit release notes. <https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/index.html>, 2024.
- [9] NVIDIA. Nvidia blackwell architecture. <https://resources.nvidia.com/en-us-blackwell-architecture>, 2024.
- [10] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Re, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels?, 2025. URL <https://arxiv.org/abs/2502.10517>.
- [11] Scaling Intelligence Lab at Stanford University. Kernelbench v0.1. <https://scalingintelligence.stanford.edu/blogs/kernelbenchv01/>, 2025.
- [12] Kimi Team et al. Kimi linear: An expressive, efficient attention architecture, 2025. URL <https://arxiv.org/abs/2510.26692>.
- [13] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019. doi: 10.1145/3315508.3329973. URL <https://doi.org/10.1145/3315508.3329973>.
- [14] Xiaoxia Wu, Cheng Li, Reza Yazdani Aminabadi, Zhewei Yao, and Yuxiong He. Understanding int4 quantization for transformer models: Latency speedup, composability, and failure cases, 2023. URL <https://arxiv.org/abs/2301.12017>.

- [15] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule, 2024. URL <https://arxiv.org/abs/2412.06464>.

A Problem Lists

A.1 Level 1: Simple Operators

Problem	Description
1_Square_matrix_multiplication_.py	Square matrix multiplication
2_Standard_matrix_multiplication_.py	Standard matrix multiplication
3_Batched_matrix_multiplication.py	Batched matrix multiplication
4_Matrix_vector_multiplication_.py	Matrix-vector multiplication
8_Matmul_with_irregular_shapes_.py	Matmul with irregular shapes
9_Tall_skinny_matrix_multiplication_.py	Tall-skinny matrix multiplication
23_Softmax.py	Softmax activation
26_GELU_.py	GELU activation
36_RMSNorm_.py	RMSNorm
40_LayerNorm.py	LayerNorm
42_Max_Pooling_2D.py	Max pooling 2D
47_Sum_reduction_over_a_dimension.py	Sum reduction
63_conv_standard_2D_square_input_square_kernel.py	2D convolution
82_conv_depthwise_2D_square_input_square_kernel.py	Depthwise 2D convolution
95_CrossEntropyLoss.py	Cross-entropy loss

A.2 Level 2: Fused Operations

Problem	Description
6_Conv3d_Softmax_MaxPool_MaxPool.py	Conv3D with pooling chain
17_Conv2d_InstanceNorm_Divide.py	Conv2D + InstanceNorm fusion
37_Matmul_Swish_Sum_GroupNorm.py	Matmul + Swish + GroupNorm
40_Matmul_Scaling_ResidualAdd.py	Matmul + scaling + residual
46_Conv2d_Subtract_Tanh_Subtract_AvgPool.py	Conv2D + activation chain
52_Conv2d_Activation_BatchNorm.py	Conv2D + BatchNorm fusion
55_Matmul_MaxPool_Sum_Scale.py	Matmul + pooling chain
59_Matmul_Swish_Scaling.py	Matmul + Swish + scaling
66_Matmul_Dropout_Mean_Softmax.py	Matmul + dropout + softmax
73_Conv2d_BatchNorm_Scaling.py	Conv2D + BatchNorm + scaling
82_Conv2d_Tanh_Scaling_BiasAdd_Max.py	Conv2D + activation chain
85_Conv2d_GroupNorm_Scale_MaxPool_Clamp.py	Conv2D + GroupNorm fusion
86_Matmul_Divide_GELU.py	Matmul + GELU fusion
98_Matmul_AvgPool_GELU_Scale_Max.py	Matmul + pooling + GELU
99_Matmul_GELU_Softmax.py	Matmul + GELU + softmax

A.3 Level 3: Single Blocks

Problem	Description
31_VisionAttention.py	Vision attention mechanism
43_MinGPTCausalAttention.py	Causal attention (MinGPT style)
44_MiniGPTBlock.py	Full transformer block

A.4 Level 4: Novel Layers

Problem	Description
1_DeepSeek_MLA.py	Multi-head Latent Attention with LoRA KV compression
2_DeepSeek_MoE.py	MoE with grouped expert routing
3_GroupedQueryAttention.py	GQA with KV head expansion (Llama 3 style)
4_FP8_Matmul.py	FP8 E4M3 quantized matmul with tensor cores
5_MoE_GatedGEMM.py	MoE with fused gated dual GEMM (SwiGLU FFN)
6_INT4_Quantized_GEMM.py	INT4 weight-only quant with group-wise dequant fusion
7_GatedDeltaNet.py	Gated delta rule linear attention (ICLR 2025)
8_KimiDeltaAttention.py	Channel-wise gated delta attention

B Supplementary Tables

B.1 Results by GPU

GPU	Pass rate (%)	Beat baseline (%)	Beat baseline (n)	Total
B200	41.2	19.0	70	369
H100	43.6	20.6	76	369

B.2 Token and Cost Accounting

Model	Total tokens	Estimated cost (USD)
Claude Opus 4.5	4776623	35.13
GPT-5.2	2111940	27.55
Grok 4.1 Fast Reasoning	6248298	21.89
Claude Sonnet 4.5	2821243	12.57
Gemini 3 Pro	5372941	7.77
MiniMax M2.1	5983144	4.04
GLM-4.7	4192667	2.87
Kimi K2 Thinking	3035630	2.08
DeepSeek V3.2	1040464	0.29